

Procedures & Interrupts

Prof. Charan Bhaskar

Procedures

In assembly language programming, a **procedure** is a named block of code or group of instructions that performs a specific task or function. Procedures are used to organize code, make it more readable, and promote reusability. They are similar to functions or methods in high-level programming languages. They make your code easier to understand, use less repetitive code, and help you organize information better. They can take input, do something with it, and provide an output. Procedures are like building blocks for your program that make it work smoothly and efficiently.

The **CALL** instruction links to the procedure, and the **RET** (return) instruction returns from the procedure. The stack stores the return address whenever a procedure is called during the execution of a program. The CALL instruction pushes the address of the instruction following the CALL (return address) on the stack. The RET instruction removes an address from the stack so the program returns to the instruction following the call. The CALL instruction differs from the jump instruction because a CALL saves a return address on the stack. address from the stack so the program returns to the instruction following the CALL. There are two types of CALL instructions:

1. **NEAR** call used to a procedure located within the same segment. It covers a distance of $\pm 32K$ in 8086 through the 80286 processors, and $\pm 2G$ for 80386 and above.
2. **FAR** call used to call a procedure in a different segment. It can call a procedure stored in any memory location.

Sample Program

Program to find the cumulative sum until 5 and store it in result variable.

```
.386
.model flat, stdcall
.stack 4096
ExitProcess proto, dwExitCode:dword

.data
result WORD ?
.code
cuml_sum proc uses ecx, ebx
    mov ecx, ebx    ; Load counter with input
    mov ax, 0       ; Initialize AX with 0
    sum_loop:
        add ax, cx   ; Add CX (counter) to AX (accumulator)
        loop sum_loop ; Decrement counter and loop if not zero
    mov result, ax   ; Store the result in 'result' variable
cuml_sum endp
```

```

        ret
cuml_sum endp
main proc
    mov ebx, 5
    call cuml_sum ;Call the cuml_sum procedure to find the cumulative sum
    invoke ExitProcess, 0
main endp
end main

```

Interrupts

An interrupt is either a hardware-generated CALL (externally derived from a hardware signal) or a software-generated CALL (internally derived from the execution of an instruction or by some other internal event). At times, an internal interrupt is called an exception. Either type interrupts the program by calling an interrupt service procedure (ISP) or interrupt handler. The microprocessor has three different interrupt instructions that are available to the programmer: INT, INTO, and INT 3.

1. **INTs:** "INT" is a general term used to refer to software interrupts in assembly language. Software interrupts are triggered using the int instruction, and they allow a program to request specific services or perform actions provided by the operating system or other software components. These interrupts are typically identified by a unique interrupt number, and they can vary in purpose, such as file operations, keyboard input, or system calls.

Example

```

.386
.model flat, stdcall
.stack 4096

.data
result word ?
.code
main proc
    mov ah, 09h ; Function code for displaying a string
    lea dx, hello_msg ; Load the address of the message
    ; int 21h ; Invoke DOS interrupt
    int 20h ; Exit the program using INT 20h
main endp
end main

```

2. **INTO:** The "INTO" instruction in x86 assembly is used to generate a software interrupt (INT 4) if the Overflow Flag (OF) is set, indicating an arithmetic overflow during a signed operation. For instance, in a simple example:

Example:

```
.386
.model flat, stdcall
.stack 4096
.data
result word ?
.code
main proc
    mov eax, 10
    add eax, 5 ; No overflow
    into
    add eax, 2147483647 ; Adding a large number to EAX, causing overflow
    into ; If overflow occurs, generate an interrupt
main endp
end main
```

3. **INT3:** INT 3 (Breakpoint Interrupt): "INT 3" is the Breakpoint Interrupt. It is a software interrupt that is used primarily for debugging purposes. When an int 3 instruction is encountered in a program, it triggers a breakpoint exception, causing the program to pause and allowing a debugger to inspect the program's state. Programmers use INT 3 to set breakpoints in their code for debugging and analysis, and it's commonly used in software development and debugging tools.

Example

```
.386
.model flat, stdcall
.stack 4096
.data
.code
main proc
    mov ah, 9
    lea dx, message
    ; Set a breakpoint by using int 3
    int 3
    ; Continue with the program
    mov ah, 9
    int 3
    lea dx, message
```

```

    int 3
main endp
end main

```

Lab Work

- Take a reference from the above sample programs and implement the following programs.
- Attach properly commented code in the lab report
- Add flow chart for the logic implemented

Question 1:

Write a program that finds the factorial of given variable n=5 and store it into result variable.

Question 2:

Write a program to reverse a given string input_string = "Microprocessor", and store the reversed string in output_string variable.

Question 3:

Write a program that finds the cube of number present in ecx (Load ecx with 4 in the main procedure and then call the procedure to find the cube) and load result into result variable.

Note: Use procedures for the above questions

Question 4:

Run the following code and attach a screenshot of your execution. Explain your understanding of the code, execution and results.

```

.386
.model flat, stdcall
.stack 4096
ExitProcess proto, dwExitCode:dword
.data
result word 0
.code
main proc
    mov eax, 10
    mov ebx, 0
    div ebx
    mov result, ax
    invoke ExitProcess, 0
main endp
end main

```